Cheatsheet · 4 chapters on one page

MLOps cheatsheet

Every formula, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

24 key ideas | 11 formulas | 12 tables | 17 tips and traps

STEP 1 Set up and plan

1.1 Set up your computer for machine learning

Python, virtual environments, Git, VS Code, Docker and GPUs set up on macOS, Windows, Linux and ChromeOS.

KEY IDEAS

- **The toolbox** Python 3.12 or newer, terminal, .venv per project, pip or uv, Git, VS Code, later Docker and a GPU or Colab.
- **PATH** The first match in PATH runs. activate puts .venv first. python -m pip keeps pip and python together.
- **Per system** macOS and Linux: python3, .venv/bin. Windows: python or py, .venv\Scripts, execution policy. ChromeOS: Linux container, Debian steps.
- **Reproducible** pip freeze > requirements.txt, pip install -r requirements.txt. Commit requirements, ignore .venv. Never sudo pip.
- **Docker** Desktop on macOS and Windows (WSL 2), Engine on Linux, docker group, docker run hello-world.
- **GPU** NVIDIA: driver, nvidia-smi, CUDA build of PyTorch. Apple silicon: MPS. Chromebook: Colab or Kaggle.

TABLES

Table: tool · what it is · why ml needs it

Tool	What it is	Why ML needs it
Python 3.12 or newer	The language and its interpreter	NumPy, pandas, scikit-learn and PyTorch need a recent Python
Terminal	A window where you type commands	Installing, running scripts, Git, Docker
Virtual environment	A folder with its own Python and packages	Each project keeps its own versions
pip or uv	Package installers	Install libraries from PyPI, the Python package index
Git	Version control	History of code and configs, teamwork, CI
VS Code	Editor with Python and Jupyter support	Write, run and debug code and notebooks
Docker	Runs apps in containers	Ship the model with everything it needs
GPU or cloud notebook	Fast parallel hardware	Training deep networks in hours, not days

Lesson: What you need and why

Table: command · pip and venv · uv

Command	pip and venv	uv
Create an environment	`python3 -m venv .venv`	`uv venv`
Install packages	`pip install pandas`	`uv pip install pandas`
Install from a file	`pip install -r requirements.txt`	`uv pip install -r requirements.txt`
Save exact versions	`pip freeze > requirements.txt`	`uv pip freeze > requirements.txt`
Install a Python version	Use the python.org installer	`uv python install 3.14`

Lesson: Virtual environments and packages

Table: problem · system · fix

Problem	System	Fix
`python: command not found`	macOS, Linux, ChromeOS	Use `python3`, or activate the .venv
Typing python opens the Microsoft Store	Windows	Turn off the App execution aliases for python
Running scripts is disabled on this system	Windows PowerShell	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
`error: externally-managed-environment`	Debian, Ubuntu, Fedora, ChromeOS	Install inside a virtual environment, never with sudo pip
ensurepip is not available	Debian, Ubuntu, ChromeOS	`sudo apt install python3-venv`
A package installed but import fails	All	pip and python come from different places: use `python -m pip`
Linux option missing in Settings	ChromeOS	Managed device: ask the administrator or use Colab
Docker permission denied	Linux, ChromeOS	Join the docker group, then log out and back in

Lesson: GPUs, the cloud and common problems

Table: · venv and pip · uv · conda

	venv and pip	uv	conda	Docker
What it isolates	Python packages	Python packages and Python versions	Python and non Python libraries	The whole system: OS libraries, Python, packages
Comes with Python	Yes	No, one small install	No, a separate distribution	No, a separate app
Speed	Fine	Very fast	Slower	Slow to build, fast to start
Lock file	requirements.txt	uv.lock or requirements.txt	environment.yml	Dockerfile plus requirements
Best for	Learning, small projects	Everyday work, CI	Science stacks with C and CUDA libraries	Serving and deploying

Lesson: venv, uv, conda and Docker compared

TIPS AND TRAPS

REMEMBER

Run this check after each part of the setup. The output above is from the computer that built this page: Python 3.14.6 on Linux inside a virtual environment, with every package installed. Inside this web page, the same program runs in the browser's own Python, so it reports a different system and that the web packages are not installed.

Lesson: What you need and why

IN THE EXAM

Two names, one idea: macOS, Linux and ChromeOS call it `python3`, Windows calls it `python` or `py`. Inside an activated virtual environment, `python` works everywhere.

Lesson: Install Python and open a terminal

IN THE EXAM

The program shows what python -m venv creates: a bin folder (Scripts on Windows), include and lib folders, and pyvenv.cfg. The new python reports that it is inside a virtual environment, its packages go to the environment's own site-packages, and include-system-site-packages = false means it cannot see packages installed for the whole computer.

Lesson: Virtual environments and packages

REMEMBER

The check finds two problems: NumPy 2.4.6 is installed but the file asks for 2.3.5, and MLflow is missing. Running pip install -r requirements.txt fixes both, which is exactly what a CI job or a new teammate does first.

Lesson: Virtual environments and packages

REMEMBER

If hello-world prints Hello from Docker!, the install works. Permission denied on /var/run/docker.sock on Linux means you have not logged out and back in since joining the docker group.

Lesson: Docker on each system

1.2 The MLOps lifecycle and reproducible projects

The MLOps loop and maturity levels, seeds and run records, data and model versioning, and project layout.

KEY IDEAS

- **MLOps** Version, test, deploy, monitor and retrain ML. DevOps plus data and models.
- **Levels** 0 manual, 1 automated training pipeline (continuous training), 2 CI/CD for the pipeline.
- **Reproducible** Fix seeds, report spread over seeds, save run records: data fingerprint, params, metrics, versions.
- **Versions** Code in Git, data by hash (DVC), models in a registry, environment in requirements or images. Lineage.
- **Tests** Data, features, model quality and behaviour, infrastructure. Notebooks to tested functions.
- **Why it matters** Models decay as data changes. Hidden technical debt lives outside the model code.

FORMULAS

An ML system — Exam favourite

$$\text{behaviour} = \text{code} + \text{data} + \text{model} + \text{configuration}$$

All four need versions.

Lesson: What MLOps is

Continuous training — Exam favourite

$$\text{new data} \rightarrow \text{retrain} \rightarrow \text{evaluate} \rightarrow \text{register} \rightarrow \text{deploy}$$

The loop of level 1.

Lesson: What MLOps is

Automation share

$$\text{automated stages} / \text{all stages}$$

A rough maturity measure.

Lesson: What MLOps is

TABLES

Table: level · what is automated · typical sign

Level	What is automated	Typical sign
0: manual	Nothing. Notebooks, files copied by hand	A model is retrained a few times a year
1: ML pipeline automation	Data, training, evaluation and registration run as one pipeline, retraining is triggered	Continuous training (CT) on new data
2: CI/CD pipeline automation	The pipeline itself is tested, built and deployed by CI/CD	Many models updated safely every week

Lesson: What MLOps is

Table: what · versioned with · example

What	Versioned with	Example
Code and configs	Git	Commit a1b2c3d
Data	Fingerprint, DVC or lakeFS	customers.csv, hash 25e2eb7b7a87
Environment	requirements.txt, lock file, Docker image tag	ml-api:1.4.0
Model	Model registry	churn-model version 7, stage production
Experiments	Tracking server	MLflow run 3f9e, accuracy 0.981

Lesson: Versioning data and models

Table: test type · checks · example

Test type	Checks	Example
Data tests	Schema, ranges, missing values, label balance	age between 0 and 120
Feature tests	Transformations do what they claim	Scaled column has mean near 0 on training data
Model tests	Quality floor, behaviour on known cases	Accuracy at least 0.9, prediction unchanged when the name changes
Infrastructure tests	Pipeline runs end to end, model loads and serves	Training on 100 rows finishes, API returns a valid answer

Lesson: A project layout that grows into production

Table: · devops · mlops · dataops

	DevOps	MLOps	DataOps
What it ships	Software	Models and the pipelines that make them	Data products and pipelines
What changes behaviour	Code	Code, data and model	Data and transformations
Testing	Unit and integration tests	Plus data tests and model quality gates	Data quality checks
After release	Uptime and errors	Plus drift, accuracy and fairness	Freshness and quality of data
Typical tools	Git, CI/CD, Docker, Kubernetes	Plus MLflow, DVC, feature stores, registries	Airflow, dbt, data contracts

Lesson: DevOps, MLOps and DataOps compared

TIPS AND TRAPS

IN THE EXAM The first block changes every time you run it, which is the point. With the seed fixed at 42, five runs give exactly the same accuracy, 0.972, five times. Over ten different seeds the accuracy has mean 0.956 and standard deviation 0.014, ranging from 0.937 to 0.972, so a difference of 0.01 between two models trained once means nothing.

Lesson: Reproducibility: seeds and run records

REMEMBER The record says the wine data had 178 rows and 13 features with fingerprint 3a28849ef236, lists the parameters (C = 0.5, seed 7), the metrics (test accuracy 0.9778) and the Python, NumPy and scikit-learn versions. With this one JSON file, anyone can rebuild and check the model.

Lesson: Reproducibility: seeds and run records

IN THE EXAM The original breast cancer data gives a 5 fold accuracy of 0.981. Flipping 40 labels (7 percent) drops it to 0.882 with the same number of rows. Losing about 30 percent of the rows (399 left) gives 0.975. Each version has a different fingerprint, so the run record shows exactly which one a model saw.

Lesson: Versioning data and models

STEP 2 Build and ship

2.1 Pipelines and saving models

Leak free scikit-learn pipelines, saving with joblib and metadata, file formats, and input contracts.

KEY IDEAS

- **Pipeline** Preprocessing plus model in one object. ColumnTransformer: numbers impute and scale, categories one hot.
- **fit and transform** Steps learn on training rows only, reuse at predict. handle_unknown='ignore' for unseen categories.
- **Leakage** No step may see test rows. Selection outside CV: 0.85 on noise, inside: 0.48. Split by time and group too.
- **Saving** joblib.dump the whole pipeline plus JSON metadata (features, classes, versions). Load trusted files only.
- **Formats** joblib for own Python services, skops for safe sharing, ONNX for other languages.
- **Input contract** Check presence, type, range, unknown categories, extra fields. Reject with 422.

FORMULAS

Standard scaling Exam favourite

$$z = (x - \mu_{train}) / \sigma_{train}$$

Statistics from the training data only.

Lesson: One pipeline from raw columns to prediction

Median imputation Exam favourite

$$\text{missing } x \rightarrow \text{median of training } x$$

Robust to outliers.

Lesson: One pipeline from raw columns to prediction

One hot columns Exam favourite

$$\text{one column per category seen in training}$$

Unknown category → all zeros with ignore.

Lesson: One pipeline from raw columns to prediction

TABLES

Table: kind of leakage · example · fix

Kind of leakage	Example	Fix
Preprocessing on all rows	Scaler fitted before the split	Scaler inside the pipeline
Selection on all rows	Top correlated columns chosen with test rows	SelectKBest inside the pipeline
Target leakage	A column filled in after the outcome is known	Remove it, check the time each column becomes available
Time leakage	Random split of time ordered data	Split by time: train on the past, test on the future
Group leakage	The same patient in train and test	GroupKFold by patient

Lesson: Leakage: why every learning step belongs inside the pipeline

Table: · joblib (pickle) · skops · onnx

	joblib (pickle)	skops	ONNX	Framework format
What it stores	Any Python object	scikit-learn objects	A computation graph	PyTorch state_dict, Keras .keras
Safe to load from strangers	No, can run code	Yes, checks types first	Yes	Mostly (PyTorch weights_only loading)
Needs Python to serve	Yes, same versions	Yes	No: C++, C#, Java, JavaScript runtimes	Usually
Speed of serving	Normal	Normal	Often faster with ONNX Runtime	Normal
Best for	Your own models, quick start	Sharing scikit-learn models	Serving across languages and devices	Deep learning models

Lesson: Ways to save a model compared

TIPS AND TRAPS

IN THE EXAM

The churn pipeline reaches a 5 fold ROC AUC of 0.751 and a test accuracy of 0.69. After preprocessing, 5 raw columns have become 9 (3 scaled numbers and 6 one hot columns, with names such as cat__plan_basic). A new customer with a missing bill and a plan called student, which the model never saw, still gets a churn probability of 0.782 instead of an error.

Lesson: One pipeline from raw columns to prediction

IN THE EXAM

On 100 rows of random noise with random labels, choosing the 20 best of 5000 columns before cross validation reports an accuracy of 0.85. Doing the same selection inside the pipeline reports 0.48, the honest answer for labels that are pure chance.

Lesson: Leakage: why every learning step belongs inside the pipeline

IN THE EXAM

The fitted iris pipeline takes about 1.5 KB on disk. After loading, its predictions match the original on every row, and the flower [5.9, 3.0, 5.1, 1.8] is predicted as virginica. The JSON file records scikit-learn 1.8.0, which the serving side should match.

Lesson: Saving and loading a trained pipeline

REMEMBER

The first request passes. The second is rejected because tenure is −3 and the bill was sent as text. The third has four problems: a missing bill, 2.5 support calls where a whole number is required, an unknown plan, and an unexpected coupon field.

Lesson: An input contract for the model

2.2 Serving models with Flask, FastAPI and Django

Model APIs in Flask, FastAPI and Django, calling them on each OS, online against batch, and production servers sized with Little's law.

KEY IDEAS

- **Model API** Load once. /health plus POST /predict. JSON in and out. 200 success, 422 bad content, 405 wrong method, 500 is a bug.
- **Frameworks** Flask small. FastAPI checks Pydantic classes, docs at /docs. Django for a model inside a full web app.
- **Test then call** Test client in one process. Then curl (macOS, Linux, ChromeOS) or Invoke-RestMethod and curl.exe (Windows).
- **Online or batch** Online answers during the visit. Batch scores on a schedule, cheaper, less fresh. Read big files in chunks.
- **Production servers** Gunicorn on macOS and Linux, Waitress on Windows, Uvicorn for FastAPI everywhere. Several workers. Never app.run in production.
- **Capacity** L = λW. Workers = λ × latency / target use. M/M/1: W = 1/(μ − λ) explodes near capacity.

FORMULAS

Latency Exam favourite

time from request sent to response received

Report the median and the 95th or 99th percentile, not only the average.

Lesson: A model behind a web API

Throughput of one worker Exam favourite

requests per second = 1000 / latency in ms

A worker that needs 20 ms serves at most 50 requests per second.

Lesson: A model behind a web API

Little's law Exam favourite

$$L = \lambda \times W$$

L requests in progress, λ arrivals per second, W seconds each request spends in the system.

Lesson: From the development server to production

Workers needed Exam favourite

workers = λ × latency / target use

Aim for about 70 to 80 percent use so short bursts do not build a queue.

Lesson: From the development server to production

Waiting time for one worker (M/M/1 queue)

$$W = 1 / (\mu - \lambda)$$

μ requests per second the worker can serve. W shoots up as λ nears μ.

Lesson: From the development server to production

TABLES

Table: status · meaning · when a model api sends it

Status	Meaning	When a model API sends it
200 OK	Success	The prediction is in the body
400 Bad Request	The request itself is broken	The body is not valid JSON at all
405 Method Not Allowed	Wrong method for this route	GET sent to a POST only route
422 Unprocessable Content	Valid JSON but the content fails the checks	A word instead of a number, a missing field
500 Internal Server Error	A bug or crash inside the service	Never on purpose. Log it and fix it.

Lesson: A model behind a web API

Table: · flask · fastapi · django

	Flask	FastAPI	Django	Model servers (BentoML, TorchServe, KServe)
Size	Small and simple	Small, typed	Large, everything included	A platform
Input checks	Write them yourself	Automatic from Pydantic classes	Forms or Django REST framework serializers	Built in per model
API docs	Add a package	Automatic at /docs	Add a package	Usually automatic
Production server	Gunicorn, Waitress	Uvicorn	Gunicorn, Waitress	Included
Extras	Many plugins	Async, dependency injection	Database, admin, users	Batching requests, GPUs, scaling, model versions
Best for	Learning, small services	New model APIs	A model inside a larger web app	Many models or heavy deep learning traffic

Lesson: Flask, FastAPI, Django and model servers compared

TIPS AND TRAPS

REMEMBER

The test client sends real requests to the app inside one Python process, so no port or network is needed. The good request gets 200 with virginica at 0.703. The broken one gets 422 and an error that tells the caller what to send. The model never sees bad data.

Lesson: A model behind a web API

IN THE EXAM

One bad request produced three precise errors at once: sepal_length must be greater than 0, sepal_width is not a number, and petal_width is missing. The batch route scored two flowers in a single predict_proba call, virginica at 0.703 and setosa at 0.981.

Lesson: FastAPI: the input contract written as a class

COMMON MISTAKE

Django gives the same answers as Flask (200 with virginica at 0.703, 422 for bad JSON), and @require_POST turns a GET into 405 without running the view. In a real project the view lives in an app inside `django-admin startproject`, and the settings come from environment variables, never a SECRET_KEY written in the code.

Lesson: Django: when the model joins a full web application

REMEMBER

20,000 rows are scored in 4 chunks of 5,000. The last line times both styles on your own device, so the numbers change from run to run, but one call per row is always many times slower because each call pays the same fixed overhead.

Lesson: Online against batch prediction

IN THE EXAM

A queue does not grow in a straight line. One worker that can serve 50 requests per second answers in 25 ms at 10 per second, 100 ms at 40 per second, and 1 full second at 49 per second. That is why services keep spare capacity.

Lesson: From the development server to production